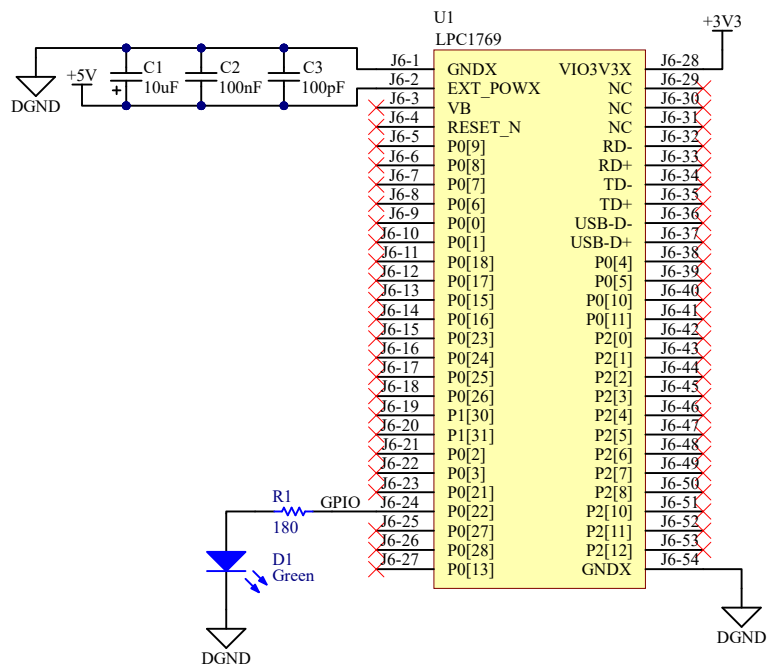


```

1  /*=====
2  * @file    Ejercicio_5_1_1.c
3  *
4  * @brief    Controla un LED conectado al pin P0.22, alternando periódicamente
5  *           entre los estados encendido y apagado, con un periodo de intermitencia
6  *           de 200ms.
7  *
8  * @details  - Implementa la temporización mediante interrupciones periódicas
9  *           (no bloqueante), vía TMR0 cada 100 ms.
10 *           - Implementación en Bajo nivel: Acceso directo a registros mediante
11 *             punteros a estructuras CMSIS del fabricante (NXP LPC17xx).
12 *
13 * @author   Ing. E. Migliore
14 *
15 * @version  3.0
16  =====*/

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5  #include <stdbool.h>
6
7  /*=====
8  * DEFINICIÓN DE CONSTANTES
9  *=====*/
10 /* Bits físicos de prioridad (0 al 31) */
11 #define NVIC_PRIO_BITS      5U
12
13 /* Identificadores de Prioridad de IRQ */
14 #define TMR0_PRIORITY      0U
15
16 /* Identificadores de IRQ */
17 #define TMR0_IRQ          1U
18
19 /* Valores de recarga para TMR0 */
20 #define N_TICKS_100MS      99U      // (100 ms / 1 ms) - 1 = 99 cuentas (0 a 99)
21
22 // #define CCLK_FREQ_HZ      100000000U
23 // #define PCLK_DIV          4U
24 // #define PCLK_FREQ_HZ      (CCLK_FREQ_HZ / PCLK_DIV)
25 // #define PR_1MS_VALUE      25000U
26
27 // #define T_MATCH_MS        100U
28
29 /* MR = [ (t_match_ms * PCLK_FREQ) / (1000 * PR_VALUE) ] - 1 */
30 // #define N_TICKS_100MS      (((T_MATCH_MS * (PCLK_FREQ_HZ / 1000U)) / PR_1MS_VALUE) - 1U)
31
32 /* Control de Alimentación de Periférico */
33 #define TMR0_CTRL_ON        true
34 #define TMR0_CTRL_OFF      false
35
36 /*=====
37 * DEFINICIÓN DE VARIABLES GLOBALES
38 *=====*/
39
40 /*=====
41 * PROTOTIPOS DE FUNCIONES
42 *=====*/
43 static inline void cfgNVIC(int32_t, uint32_t);
44
45 void cfgGPIO(void);
46 void cfgTMR0(uint32_t nTicks, uint32_t ctrlTMR0);
47
48 /*=====
49 * FUNCIÓN PRINCIPAL
50 *=====*/
51 int main(void)
52 {
53     /*-----
54     * INICIALIZACIÓN DEL MCU
55     *-----*/
56     cfgGPIO();
57     cfgTMR0(N_TICKS_100MS, TMR0_CTRL_ON);
58
59     /*-----
60     * INICIO PROGRAMA PRINCIPAL
61     *-----*/
62     while(1){}
63
64     return 0;
65 }
66
67 /*=====
68 * DEFINICIONES DE FUNCIONES
69 *=====*/

```

```

70 /*****
71 * @brief Configura el NVIC y SCB para la habilitación de interrupciones.
72 *
73 * @param <irq_number> Número de IRQ en la tabla del vector.
74 * @param <priority_number> Nivel de prioridad del vector.
75 *
76 * @return None
77 *****/
78 static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
79 {
80     //Asigna Prioridad de Interrupción
81     if(irq_number < 0) {
82         //Ejecuta interrupciones del sistema.
83         //SHP = System Handler Priority Registers
84         SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((priority_number << (8 -
            NVIC_PRIO_BITS)) & 0xFF);
85     } else {
86         //Ejecuta interrupciones de periféricos.
87         //IP = Interrupt Priority Registers
88         NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
            0xFF);
89     }
90
91     //Habilitación de Interrupción
92     NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
93 }
94
95 /*****
96 * @brief Configura las direcciones GPIO, modos de función y resistencias.
97 *
98 * @details - P0.22: Salida digital.
99 *           - P1.28: Canal MAT0.0.
100 *
101 * @param None
102 *
103 * @return None
104 *****/
105 void cfgGPIO(void)
106 {
107     LPC_PINCON->PINSEL1 &= ~(0b11 << 12); //P0.22 con Funcionalidad GPIO
108     LPC_PINCON->PINSEL3 |= (0b11<<24); //P1.28 con Funcionalidad MAT0.0 (TMR0)
109
110     LPC_GPIO0->FIODIR |= (1<<22); //P0.22 como Salida Digital
111
112     return;
113 }
114
115 /*****
116 * @brief Configura e inicializa el periférico TIMER0 en modo Timer.
117 *
118 * @details Modulo configurado con base de tiempo del Prescaler en 1 ms
119 *           y generación de interrupción periódica por Match 0 (MR0) con Reset.
120 *
121 * @param <nTicks> Valor limite del registro de coincidencia MR0.
122 * @param <ctrlTMR0> Estado de alimentación del periférico (true: ON / false: OFF).
123 *
124 * @return None
125 *****/
126 void cfgTMR0(uint32_t nTicks, bool ctrlTMR0)
127 {
128     //Estado de alimentación del periférico
129     if (ctrlTMR0){
130         LPC_SC->PCONP |= (1 << 1); //Encendido del Periférico (TMR0 ON) - [Default: ON]
131     }else{
132         LPC_SC->PCONP &= ~(1 << 1); //Apagado del Periférico (TMR0 OFF)
133     }
134     //PCLK_TIMER0 = CCLK/4
135     LPC_SC->PCLKSEL0 &= ~(0x03 << 2);
136     //Timer Mode (Incremento de TC por Match de PR) - [Default: 00]

```

```

137     LPC_TIM0->CTCR &= ~(3 << 0);
138     //Valor Máximo para PC
139     LPC_TIM0->PR = 25e6*(1000/1e6);
140     //Habilitación Reset del TIMER0: (Counter Reset=1)
141     LPC_TIM0->TCR |= (1 << 1);
142     //Deshabilitación Reset del TIMER0: (Counter Reset=0)
143     LPC_TIM0->TCR &= ~(1 << 1);
144     //Limpia Bandera de Interrupción por MATCH0
145     LPC_TIM0->IR |= (1 << 0);
146
147     //Valor Máximo para TC
148     LPC_TIM0->MR0 = nTicks;
149     //Habilitación de Interrupción por MATCH: (MR0I=1)
150     LPC_TIM0->MCR |= (1 << 0);
151     //Habilitación de Reset por MATCH: (MR0R=1)
152     LPC_TIM0->MCR |= (1 << 1);
153     //Deshabilitación de Stop por MATCH: (MR0S=0)
154     LPC_TIM0->MCR &= ~(1 << 2);
155     //Modalidad Ninguna en P1.28: (EMC0=00)
156     LPC_TIM0->EMR &= ~(0x03 << 4);
157
158     //Habilitación del TIMER0: (Counter Enable=1)
159     LPC_TIM0->TCR |= (1 << 0);
160
161     //Habilitación de Interrupción por TIMER0
162     cfgNVIC(TMR0_IRQ, TMR0_PRIORITY);
163
164     return;
165 }
166
167 /*****
168  * @brief Rutina de Servicio de Interrupción (ISR) para el vector TIMER0.
169  *
170  * @details Alterna el estado lógico del pin P0.22 en cada desborde (100 ms)
171  * mediante los registros FIOSET y FIOCLR, logrando una onda cuadrada
172  * con período total de 200 ms.
173  *
174  * @param None
175  *
176  * @return None
177  *****/
178 void TIMER0_IRQHandler(void)
179 {
180     static uint8_t vTicks = 0;
181
182     //Consulta Interrupción por MR0 (MATCH0 del Timer0)
183     if(LPC_TIM0->IR & (1 << 0)){
184         //Ejecución de la ISR
185         vTicks++;
186         if(vTicks%2){
187             LPC_GPIO0->FIOSET = (1 << 22);
188         }
189         else{
190             LPC_GPIO0->FIOCLR = (1 << 22);
191         }
192
193         //Limpia Bandera de Interrupción por TIMER0
194         LPC_TIM0->IR |= (1<<0);
195
196     }
197
198     return;
199 }
200

```